МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для управления проектами и отслеживания задач»

Выполнил: Набиуллин Артур Фанилевич
Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /
«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Набиуллину Артуру Фанилевичу,

обучающемуся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для управления проектами и отслеживания задач»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	05.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	11.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	25.04.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования)	7	02.05.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	09.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Реализация подсистем аутентификации и авторизации пользователей	8	21.05.2026	
Задача 3.2. Реализация системы распределения ролей и проверки прав доступа к функциям приложения	7	26.05.2026	
Задача 3.3. Реализация CRUD-интерфейсов для управления проектами, задачами и профилями	7	30.05.2026	
Задача 3.4. Разработка функциональности загрузки и скачивания файлов, прикрепленных к задачам	12	03.06.2026	
Задача 3.5. Реализация модуля агрегирования данных для расчёта и вывода статистики по проектам	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	09.06.2026	
Задача 4.2. Деплой приложения на хостинг	4	11.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания « 31 » марта 2026 г.

Дата сдачи выполненной работы « 13 » июня 2026 г.

Задание принял к исполнению

« 31 » марта 2026 г. _____ А. Ф. Набиуллин
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **5**

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ **6**

1.1 Обзор существующих программных продуктов по теме работы 6

1.2 Анализ программных инструментов разработки веб-приложений 6

1.3 Формулировка цели и задач работы 7

2 ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ **9**

2.1 Анализ целевой аудитории 9

2.2 Описание функциональности приложения 9

2.3 Проектирование модели данных 11

2.4 Разработка макетов страниц 12

3 РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ **14**

3.1 Реализация подсистемы аутентификации и проверки ролей . . 14

3.2 Разработка CRUD-интерфейсов и работы с файлами 16

3.3 Реализация модуля агрегирования данных 16

ЗАКЛЮЧЕНИЕ **18**

ВВЕДЕНИЕ

Актуальность темы. В современных условиях цифровизации бизнеса и повсеместного перехода на удаленный или гибридный формат работы, эффективное управление проектами становится критически важным фактором успеха любой организации. Использование традиционных методов учета задач (электронные таблицы, мессенджеры) приводит к потере информации, срыву сроков и снижению производительности команды. Разработка специализированных веб-приложений для трекинга задач позволяет централизовать рабочие процессы, автоматизировать контроль поручений и обеспечить прозрачность работы на всех этапах жизненного цикла проекта.

Степень разработанности проблемы. На рынке существует множество систем управления проектами (Jira, Trello, Asana и др.). Однако большинство корпоративных решений обладают избыточным функционалом, сложны в освоении и имеют высокую стоимость лицензий, что делает их нерентабельными для малого бизнеса, студенческих команд или индивидуального использования. В связи с этим, создание легковесного, интуитивно понятного веб-приложения с базовым, но достаточным набором функций (CRUD-операции, ролевая модель, прикрепление файлов) остается актуальной практической задачей.

Объект исследования — процессы управления проектами, распределения задач и контроля их выполнения в рабочих группах.

Предмет исследования — программные средства, алгоритмы и методы веб-разработки, применяемые для создания систем отслеживания задач.

Целью данной работы является проектирование и реализация клиент-серверного веб-приложения, предназначенного для эффективного управления проектами и отслеживания статусов выполнения задач с учетом ролевой модели доступа.

АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов по теме работы

На современном рынке программного обеспечения для управления проектами можно выделить несколько лидеров. Для определения требований к разрабатываемой системе проведем сравнительный анализ популярных платформ. Результаты сравнения представлены в таблице 1.1.

Таблица 1.1 – Сравнительный анализ систем управления проектами

Название	**Ключевые особенности**	**Недостатки (для малых команд)**
Jira	Мощный инструментарий для Agile-разработки, детальная настройка рабочих процессов, глубокая интеграция с CI/CD.	Перегруженный интерфейс, высокая сложность первоначальной настройки, ресурсоемкость.
Trello	Интуитивно понятный интерфейс на базе Kanban-досок, простота использования, визуализация процессов.	Ограниченные возможности для сложной аналитики и агрегации данных, слабый контроль доступа в бесплатной версии.
Asana	Гибкое представление задач (списки, доски, таймлайны), удобное управление зависимостями между задачами.	Строгие ограничения в базовом тарифе, навязчивое предложение платных подписок.

Анализ показал, что существующие решения либо слишком сложны и дороги (Jira), либо ограничивают важный функционал, такой как детальная статистика или расширенные права доступа, в базовых версиях (Trello, Asana). Это подтверждает целесообразность разработки собственного легковесного решения, объединяющего простоту интерфейса и наличие необходимых инструментов управления.

1.2 Анализ программных инструментов разработки веб-приложений

Для реализации проекта выбран подход создания веб-приложения на базе языка программирования Python. Этот выбор обусловлен высокой скоростью разработки, читаемостью кода и богатой экосистемой библиотек.

- **Frontend-часть:** Пользовательский интерфейс будет реализован с использованием классических веб-технологий. Для рендеринга динамического контента на стороне сервера выбран шаблонизатор **Jinja2**, тесно интегрированный во Flask.

- **Backend-фреймворк:** В качестве основы серверной части выбран микрофреймворк **Flask** [1]. В отличие от тяжеловесных решений (таких как Django), Flask предоставляет гибкость архитектуры, позволяя разработчику самостоятельно выбирать необходимые модули. Он отлично подходит для реализации маршрутизации, обработки HTTP-запросов, а также интеграции подсистем аутентификации (например, через Flask-Login).

- **Работа с базой данных:** Для взаимодействия с базой данных используется **SQLAlchemy** [2] — мощная библиотека объектно-реляционного отображения (ORM). Использование ORM позволяет описывать структуру таблиц (Пользователи, Проекты, Задачи) в виде классов Python, избегая написания чистых SQL-запросов. SQLAlchemy обеспечивает безопасность (защиту от SQL-инъекций), поддержку CRUD-операций и удобные инструменты для агрегации данных (расчета статистики по проектам).

1.3 Формулировка цели и задач работы

На основе проведенного анализа предметной области (п. 1.1) и выбора технологического стека (п. 1.2), главная **цель работы** заключается в создании веб-приложения для управления проектами и отслеживания задач на базе фреймворка Flask.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ целевой аудитории и спроектировать пользовательские сценарии (Use Cases);

2. Спроектировать логическую и физическую модели реляционной базы данных;

3. Разработать макеты пользовательского интерфейса (Wireframes);

4. Реализовать подсистему аутентификации пользователей и настроить систему распределения ролей;
5. Разработать программные модули (CRUD-интерфейсы) для управления сущностями «Проект», «Задача» и «Пользователь»;
6. Реализовать функционал загрузки и скачивания файлов, прикрепленных к задачам;
7. Разработать модуль агрегирования данных для вывода аналитической статистики по проектам;
8. Развернуть готовое веб-приложение на удаленном сервере (хостинге).

ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Основными пользователями разрабатываемой системы управления проектами (Task Tracker) являются небольшие рабочие группы и студенческие команды. Для обеспечения безопасности и корректного распределения обязанностей в системе реализована ролевая модель доступа (RBAC), включающая две ключевые роли:

- **Администратор (Менеджер проекта):** обладает полными правами в системе. Занимается созданием новых проектов, управлением списком пользователей и осуществляет глобальный контроль за ходом выполнения работ через аналитические дашборды.
- **Пользователь (Исполнитель):** имеет ограниченный доступ. Видит только те проекты, в которые был приглашен. Основная задача данной роли — брать задачи в работу, изменять их статус по мере выполнения и прикреплять файлы с результатами работы.

2.2 Описание функциональности приложения

Функциональные требования к веб-приложению сформированы на основе пользовательских сценариев (Use Cases). Основной бизнес-сценарий работы системы выглядит следующим образом: Администратор авторизуется в системе и создает новый проект. Внутри проекта он формирует список задач. Пользователь, заходя в систему, видит назначенную ему задачу, переводит её в статус «В процессе», а по завершении — загружает файл с результатом (например, PDF-отчет) и меняет статус на «Готово». Администратор в режиме реального времени видит изменение статистики по проекту.

На рисунке 2.1 представлена диаграмма вариантов использования (Use Case), иллюстрирующая разграничение прав доступа.

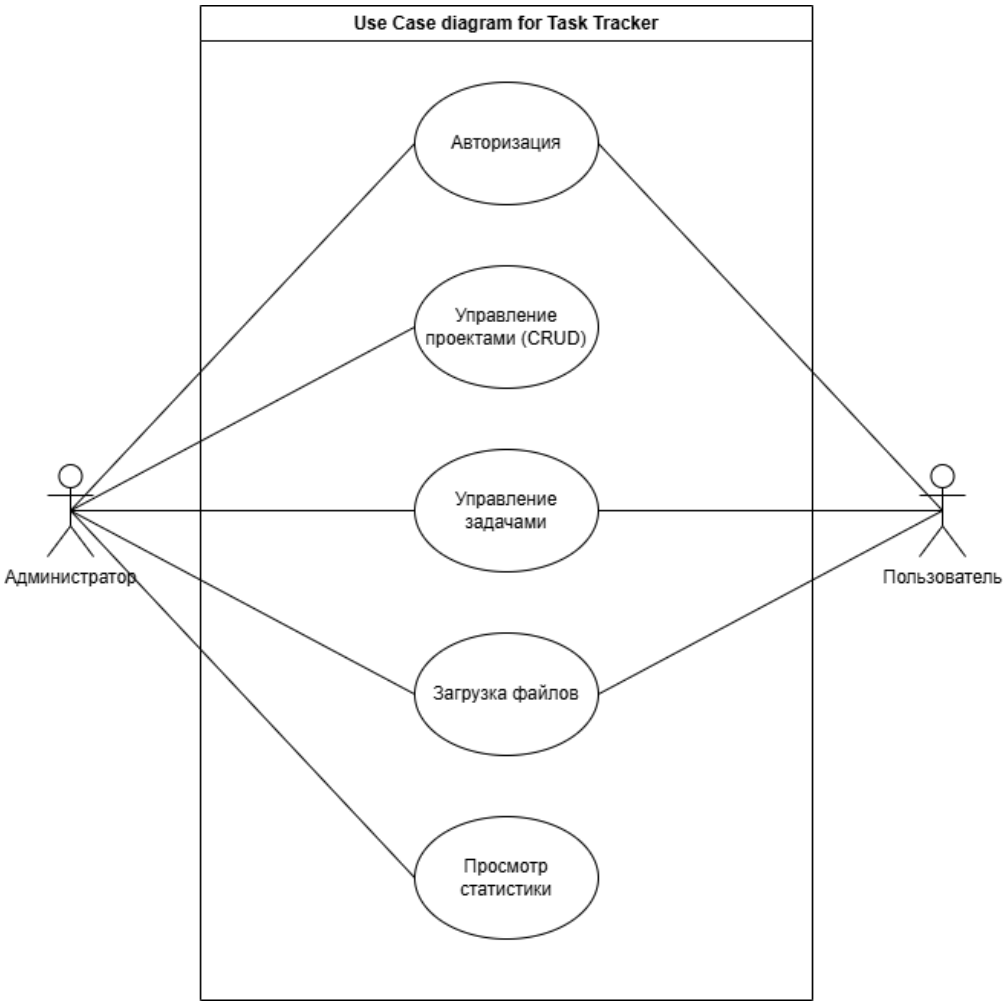

Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

Архитектура базы данных спроектирована с учетом требований нормализации [3] для исключения избыточности данных. В качестве средства взаимодействия с базой данных используется ORM-библиотека SQLAlchemy. Логическая модель данных включает три основные связанные сущности: `users` (Пользователи), `projects` (Проекты) и `tasks` (Задачи).

Структура основной таблицы задач представлена в таблице 2.1.

Таблица 2.1 – Структура таблицы задач (tasks)

Поле	**Тип**	**Ограничение**	**Описание**
id	INTEGER	PRIMARY KEY	Уникальный идентификатор задачи
title	VARCHAR	NOT NULL	Краткое название задачи
status	VARCHAR	DEFAULT 'pending'	Статус (pending, in_progress, done)
file_path	VARCHAR	NULL	Относительный путь к загруженному файлу
project_id	INTEGER	FOREIGN KEY	Ссылка на таблицу проектов
assignee_id	INTEGER	FOREIGN KEY	Назначенный пользователь (исполнитель)

Связь между таблицами реализуется посредством внешних ключей (Foreign Key), что гарантирует ссылочную целостность (например, при удалении проекта каскадно удаляются все связанные с ним задачи).

2.4 Разработка макетов страниц

Для обеспечения высокого уровня пользовательского опыта (UX) интерфейс приложения спроектирован в минималистичном стиле. Структура интерфейса позволяет пользователю получить доступ к основным функциям (добавление задачи, просмотр статистики) не более чем за два клика.

На рисунке 2.2 представлен макет панели управления (дашборда), а на рисунке 2.3 — макет детальной страницы проекта с таблицей задач.

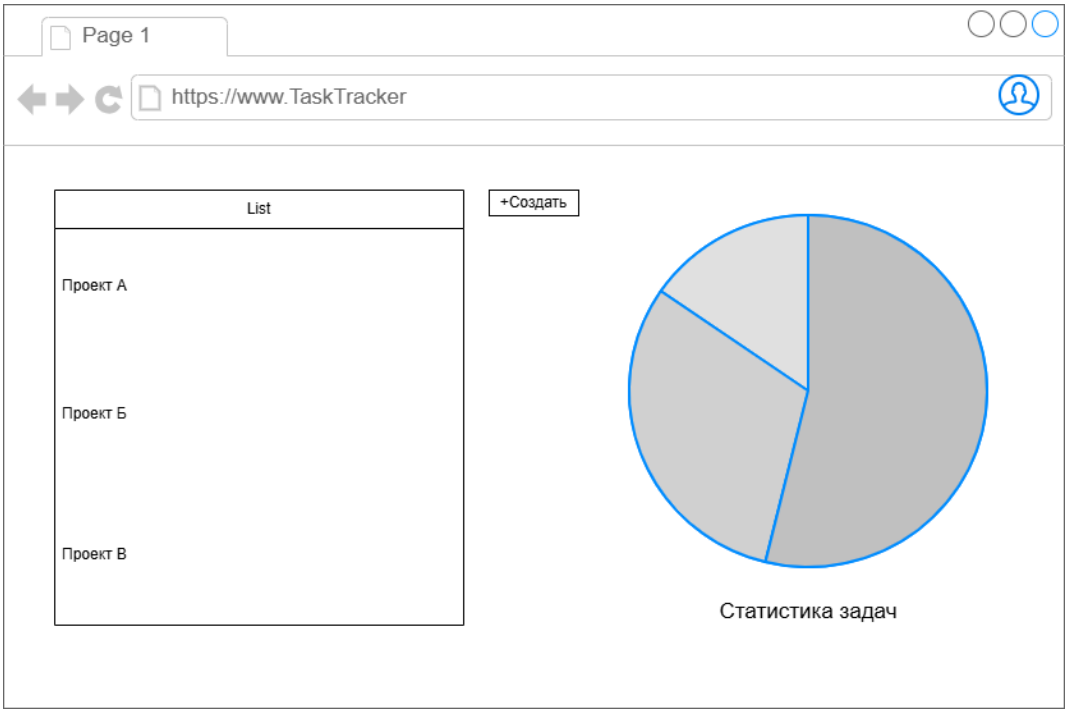

Рисунок 2.2 – Макет главной панели управления (Дашборд)

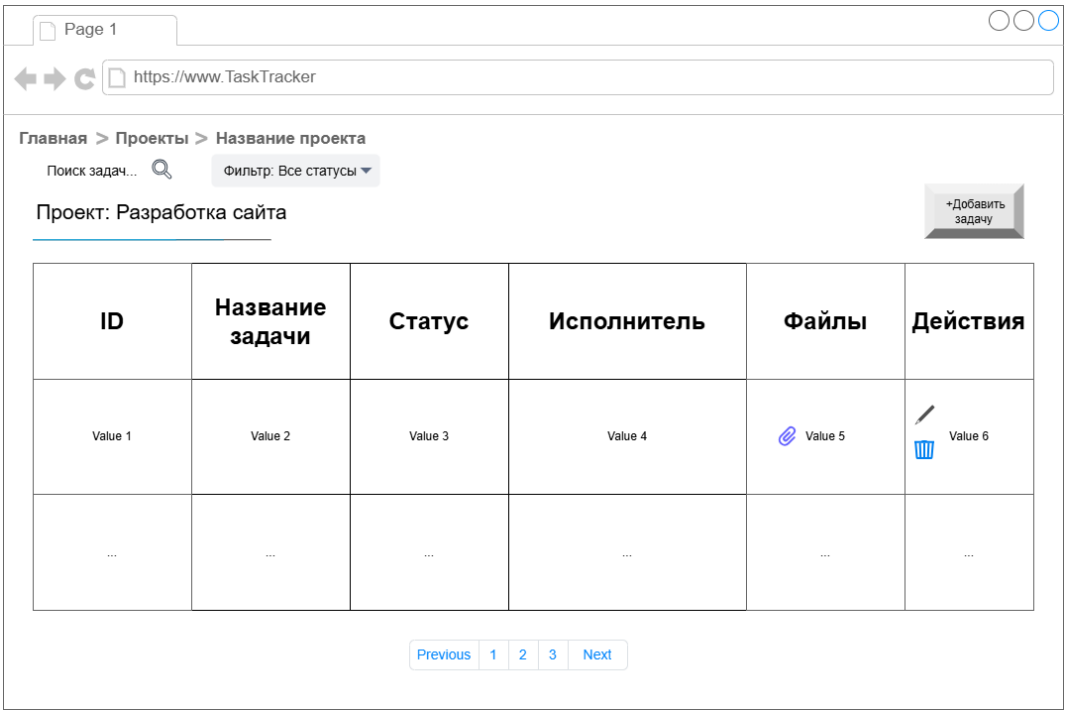

Рисунок 2.3 – Макет страницы проекта со списком задач

РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Реализация подсистемы аутентификации и проверки ролей

Основой безопасности разрабатываемого приложения является подсистема аутентификации, реализованная с помощью библиотеки `Flask-Login`. Хранение паролей в базе данных осуществляется в зашифрованном виде с использованием криптографических хэш-функций из модуля `werkzeug.security` (функции `generate_password_hash` и `check_password_hash`).

Для разграничения прав доступа была внедрена ролевая модель (RBAC). При выполнении критически важных действий (например, создание или удаление проекта) на стороне сервера происходит проверка атрибута `role` текущего пользователя.

Листинг 1 — Фрагмент кода проверки прав доступа при создании проекта

```
@app.route('/create_project', methods=['POST'])
@login_required
def create_project():
    if current_user.role != 'admin':
        flash('У вас нет прав для создания проектов.', 'danger')
        return redirect(url_for('index'))

    title = request.form.get('title')
    description = request.form.get('description')

    if title:
        new_project = Project(title=title, description=description,
                              owner_id=current_user.id)
        db.session.add(new_project)
        db.session.commit()
    return redirect(url_for('index'))
```

На рисунке 3.1 представлен интерфейс страницы входа в систему.

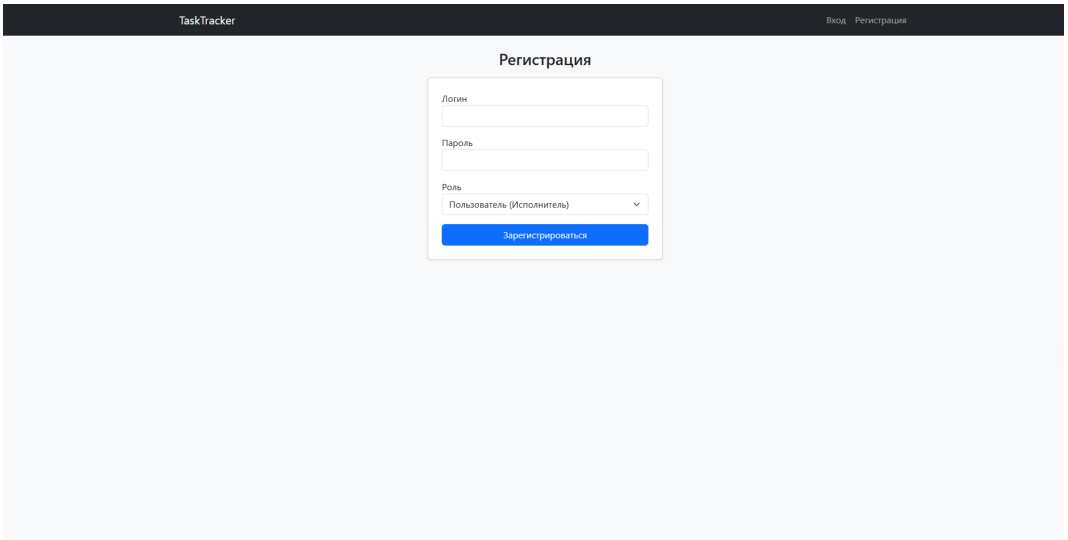

Рисунок 3.1 – Интерфейс страницы регистрации пользователей

3.2 Разработка CRUD-интерфейсов и работы с файлами

Управление сущностями системы осуществляется через реализованные CRUD-интерфейсы (создание, чтение, обновление, удаление). Взаимодействие с реляционной базой данных SQLite инкапсулировано через объектно-реляционное отображение (ORM) `SQLAlchemy`.

Особое внимание уделено функции прикрепления файлов к задачам. Для защиты файловой системы сервера от потенциально вредоносных файлов используется утилита `secure_filename`, которая нормализует имя загружаемого документа перед его сохранением в директорию `static/uploads`.

На рисунке 3.2 продемонстрирован интерфейс управления задачами внутри проекта, включающий возможности фильтрации, пагинации и управления жизненным циклом задачи.

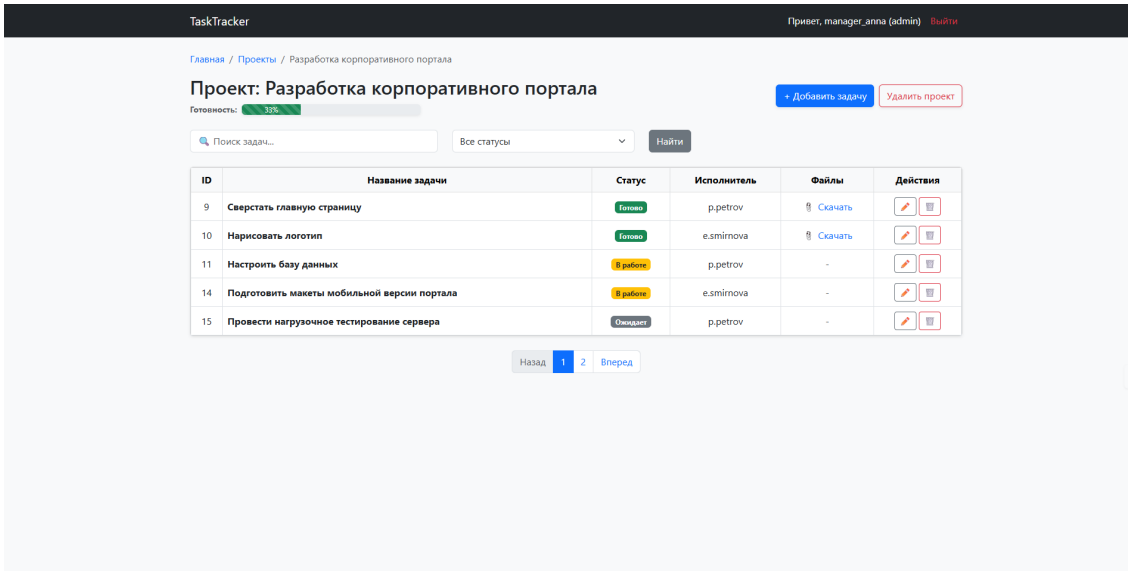

Рисунок 3.2 – Интерфейс управления задачами проекта

3.3 Реализация модуля агрегирования данных

Для обеспечения администратора системы наглядной аналитикой был разработан модуль агрегирования данных. На серверной стороне с помощью агрегатных функций SQLAlchemy (метод `.count()`) выполняется подсчет задач, сгруппированных по их текущему статусу (ожидают, в работе, выполнено).

Полученные статистические данные передаются в HTML-шаблон, где на клиентской стороне визуализируются в виде интерактивной круговой диаграммы с использованием библиотеки `Chart.js`.

Листинг 2 — Фрагмент кода агрегации данных в базе

```
# Агрегирование данных для круговой диаграммы
total_tasks = Task.query.count()
pending_tasks = Task.query.filter_by(status='pending').count()
in_progress_tasks = Task.query.filter_by(status='in_progress').count()
done_tasks = Task.query.filter_by(status='done').count()

stats = {
    'total': total_tasks,
    'pending': pending_tasks,
    'in_progress': in_progress_tasks,
    'done': done_tasks
}
```

Результат работы модуля визуализации и агрегации представлен на главной панели управления системой (рисунок 3.3).

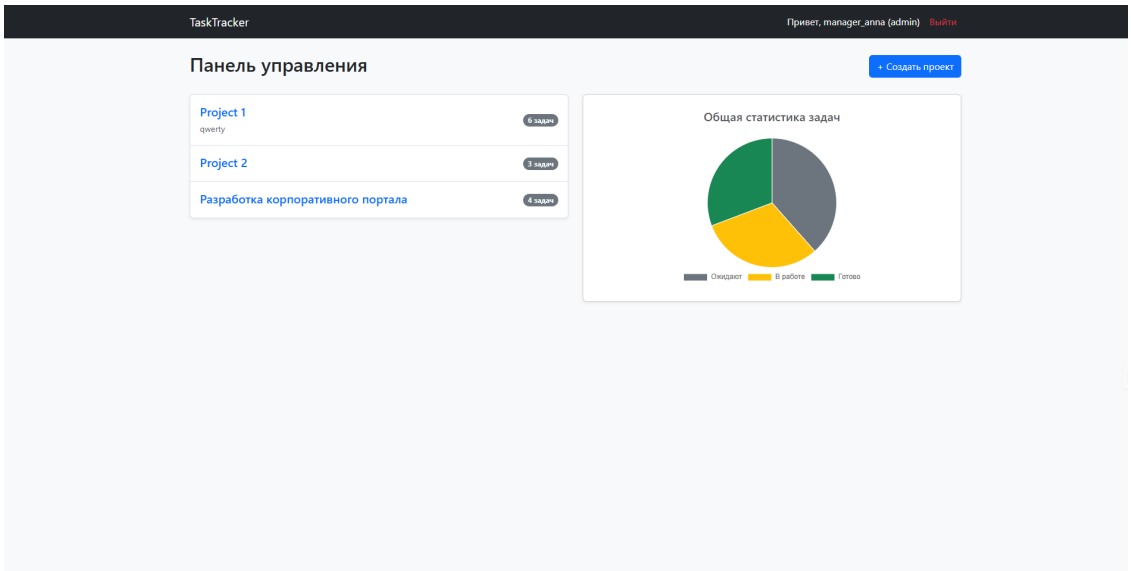

Рисунок 3.3 – Главная панель управления (Дашборд) с круговой диаграммой

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно достигнута поставленная цель — спроектировано и разработано полноценное клиент-серверное веб-приложение для управления проектами и отслеживания задач.

В результате выполнения работы были решены следующие задачи:

1. Проведен анализ предметной области, который подтвердил актуальность создания легковесных и интуитивно понятных систем управления задачами (Task Trackers) для малых рабочих групп.

2. Спроектирована архитектура приложения с использованием микрофреймворка Flask (Python) и объектно-реляционного отображения SQLAlchemy, обеспечивающая высокую производительность и безопасность взаимодействия с базой данных SQLite.

3. Разработаны пользовательские интерфейсы (Wireframes), которые впоследствии были успешно перенесены в интерактивные HTML-шаблоны с применением CSS-фреймворка Bootstrap 5.

4. Внедрена надежная подсистема аутентификации и реализована ролевая модель (RBAC), разделяющая функционал системы между «Администраторами» и обычными «Пользователями».

5. Реализован полный цикл CRUD-операций для трех ключевых сущностей предметной области: Пользователи, Проекты и Задачи.

6. Интегрирован функционал безопасной загрузки, хранения и скачивания файлов-отчетов, прикрепляемых к конкретным задачам.

7. Разработан аналитический модуль агрегирования данных, который в реальном времени визуализирует статистику выполнения задач с использованием графической библиотеки Chart.js.

Разработанная система полностью удовлетворяет техническим требованиям, отличается высокой скоростью отклика, защищенностью (шифрование паролей, защита от SQL-инъекций) и адаптивным дизайном, и может служить надежным инструментом для автоматизации рабочих процессов в малых и средних командах.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования [4], определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

[1] Flask Documentation (3.0.x) [Электронный ресурс] [Text]. — URL: https://flask.palletsprojects.com/ (дата обращения: 01.04.2026).

[2] SQLAlchemy 2.0 Documentation[Электронный ресурс] [Text]. — URL: https://docs.sqlalchemy.org/ (дата обращения: 01.04.2026).

[3] Коннолли Т. и Бегг, К. Базы данных. Проектирование, реализация и сопровождение. Теория и практика [Text] / К. Коннолли, Т. и Бегг. — [S. l.] : Вильямс, 2017. — P. 1440.

[4] ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.